

Reflexión individual — Evidencia 1

Diego Contreras Alvarez · Matrícula A00845716

Programación de Estructuras de Datos y Algoritmos Fundamentales (TC1031), grupo 607

Profesor: David Alonso Cantú Delgado · 20 de septiembre de 2026

1. Por qué importa ordenar y buscar en logs

Un log de seguridad es el diario de lo que pasa en un servidor. Cada renglón dice cuándo ocurrió algo, desde qué dirección IP y de qué se trató. Por sí solo no sirve de mucho: lo que le sirve a alguien de seguridad es poder ver los eventos en el orden en que pasaron, porque así se puede reconstruir la historia. Si primero hubo intentos de contraseña fallidos, luego un acceso no autorizado y después una fuga de datos, ese orden es justo lo que dice que fue un ataque y no tres cosas sueltas.

Un log real casi nunca llega perfectamente ordenado. Normalmente hay varios servidores escribiendo al mismo archivo, sus relojes están un poco desfasados entre sí, y algunos registros llegan tarde por la red. Por eso termina casi ordenado, con unos cuantos renglones fuera de lugar. Eso se nota en los dos archivos de esta actividad: traen exactamente los mismos 6,818 registros, pero uno viene revuelto y el otro casi en orden.

Buscar sin ordenar primero tampoco es opción. Si alguien quiere ver qué pasó entre dos horas de un día, recorrer el archivo renglón por renglón significa revisarlos todos. Con 6,818 no se nota, pero un servidor real genera millones al día. Teniendo los datos ordenados, una búsqueda binaria encuentra el pedazo que interesa en unos veinte pasos en vez de millones. Esa es la diferencia entre una consulta instantánea y una que tarda minutos.

2. Comparación de tres algoritmos

Implementé ocho algoritmos, pero los tres que más me enseñaron fueron insertion sort, merge sort y quick sort, porque entre los tres cuentan toda la historia.

Insertion sort es el que más depende de cómo vengan los datos. Su trabajo no se mide por cuántos elementos hay, sino por cuántos están fuera de lugar. En el archivo desordenado tardó 126.185 milisegundos, y en el casi ordenado 0.962, o sea 131 veces menos. Es la prueba de que un algoritmo $O(n^2)$ puede ser el más rápido de todos si los datos le convienen, y de hecho en el archivo casi ordenado solo bubble sort le ganó.

Merge sort es lo contrario: siempre hace lo mismo. Divide a la mitad pase lo que pase, así que sus tiempos fueron 3.891 y 3.032 milisegundos, casi iguales en los dos archivos. No aprovecha que los datos vengan casi ordenados, pero tampoco se puede caer. Es el que yo recomendaría para un caso real, porque es predecible y además es estable.

Quick sort fue el más interesante. Con la mediana de tres, que es la versión que hicimos en la Actividad 1.5, fue el más rápido de todos: 0.953 y 0.675 milisegundos. Pero programé aparte una variante que toma como pivote el último elemento, solo para comparar, y ahí se vio el problema. Esa variante pasó de 0.966 milisegundos en el archivo desordenado a 37.974 en el casi ordenado, casi 40 veces más lenta justo con los datos que uno pensaría que son más fáciles. La razón es que cuando los datos ya vienen en orden, el último elemento es casi siempre el mayor del pedazo, así que la partición deja todo de un lado y nada del otro. En vez de partir el problema a la mitad, lo reduce de uno en uno, y ahí cae en su peor caso $O(n^2)$. Eso me dejó claro para qué sirve la mediana de tres: no es un adorno, es lo que evita exactamente ese escenario.

También comparé la estabilidad, que es si dos registros con la misma fecha conservan el orden en que venían. Insertion y merge salieron estables, con cero diferencias contra la salida de bubble sort. Quick, selection, shell y swap no. Verifiqué que las líneas que cambiaron de lugar eran únicamente las de las 18 fechas repetidas que traen los archivos. Esto importa más de lo que parece en un log: si dos eventos quedaron grabados en el mismo segundo, un algoritmo inestable los puede invertir y hacerte creer que el ataque ocurrió antes que la alerta.

3. Mis mediciones contra mi predicción y contra la teoría

De las 16 predicciones que hice, 12 coincidieron y 4 no. Pero lo que más aprendí no fueron las que acerté, sino dos resultados que no cuadraron con lo que dice el libro.

El primero fue que insertion sort salió más lento que bubble sort en el archivo desordenado, 126.185 contra 105.697 milisegundos. La teoría dice lo contrario, porque insertion hace aproximadamente la mitad del trabajo. Tardé en entender por qué, hasta que me fijé en qué guarda cada registro: la línea completa del log como texto. Bubble sort mueve datos con `std::swap`, que en un texto solo intercambia dos apuntadores y cuesta lo mismo sin importar qué tan larga sea la línea. Insertion sort los mueve con una asignación, que copia el texto entero carácter por carácter. Como insertion hace unos 11.6 millones de movimientos en ese archivo, ese costo extra por movimiento se come toda su ventaja en comparaciones. Para comprobarlo ordené exactamente las mismas fechas pero guardadas como números simples, sin el texto, y ahí sí insertion tardó 6.619 milisegundos contra 31.385 de bubble, que es el resultado que predice la teoría.

El segundo fueron mis cuatro predicciones fallidas, que resultaron ser todas el mismo error. En selection sort en los dos archivos, en swap sort en el casi ordenado y en la variante de quick sort en el casi ordenado predije lento y salió medio. O sea que sí identifiqué cuáles iban a ser los tardados, porque son los $O(n^2)$; lo que fallé fue el umbral. Esos algoritmos hacen millones de comparaciones pero poquísimos movimientos, y comparar dos números long long es baratísimo comparado con copiar una línea de texto.

La conclusión que me llevo de los dos casos es la misma. La notación O grande dice cómo crece el trabajo conforme crecen los datos, y eso lo hace muy bien. Pero supone que todas las operaciones cuestan igual, y en la práctica no es cierto. Dos algoritmos que en el papel son los dos $O(n^2)$ pueden diferir cinco veces en tiempo real solo por cómo mueven los datos. Para estimar un tiempo de verdad hay que saber también qué tan caro sale cada operación con el tipo de dato que se está ordenando.

4. Uso de inteligencia artificial

Usé Claude durante todo el proyecto y Copilot dentro de VS Code. Desde el inicio me puse dos reglas: no aceptar código que no pudiera explicar, y no cambiar los algoritmos de clase por versiones supuestamente mejores.

El primer prompt clave fue pedirle que desglosara las instrucciones del profesor y armara un plan de trabajo por fases, con la condición de que cada fase terminara con algo que ya funcionara. Ese plan fue lo que ordenó todo el proyecto y lo que hizo que mi historial de commits muestre avance real en vez de un solo commit gigante al final. Lo usé tal cual.

El segundo fue en VS Code: escribí un comentario describiendo lo que debía hacer la función que lee un número validado, y dejé que Copilot lo completara. Me dio una versión que leía con `cin` y validaba el rango. La estructura me sirvió, pero la cambié completa a `getline`, porque mezclar `cin` con `getline` deja un salto de línea pendiente y el programa se brincaba la pregunta de la razón de la predicción. También le agregué un límite de nueve dígitos antes de convertir el texto a número, porque probando a propósito con 9999999999 descubrí que el programa se cerraba: `stoi` lanza una excepción con un número que no cabe en un `int`.

El tercero fue pedirle implementar los algoritmos que me faltaban a partir de los de mi Actividad 1.5. Ahí me propuso mejorar quick sort con un pivote aleatorio. Lo rechacé, porque la rúbrica pide los algoritmos vistos en clase; conservé la mediana de tres de mi actividad y agregué el pivote al final como una variante aparte, claramente identificada, únicamente para comparar.

Además de esos cambios, otras decisiones que tomé yo fueron quitarle los acentos a todos los mensajes de consola para que se vean igual en cualquier terminal, y agregar la validación de días por mes con años bisiestos, porque la versión que me habían sugerido aceptaba cualquier día del 1 al 31 y dejaba pasar fechas imposibles como Feb 30 o Apr 31. Ese caso terminó siendo una de mis pruebas documentadas.

Un error real de la IA y cómo lo detecté

El error más grave que encontré fue de lógica. En una versión temprana, la opción de buscar por rango estaba disponible desde el arranque, sin exigir que los datos estuvieran ordenados. Eso es un error serio y silencioso: una búsqueda binaria sobre datos desordenados no truena ni avisa, simplemente devuelve resultados falsos, porque la binaria da por hecho que si un elemento es menor que el del centro, todo lo que busca está a la izquierda. Lo detecté probando el flujo como si fuera un usuario nuevo, entrando directo a la opción 2 sin haber ordenado nada. Lo arreglé con una bandera que bloquea la búsqueda hasta que se haya ordenado y verificado, y esa bandera se vuelve a apagar cada vez que se carga un archivo nuevo, para que no se pueda buscar sobre datos que ya no corresponden.

El segundo error fue una predicción equivocada. Al pedirle los algoritmos, la IA me dio una tabla de tiempos esperados donde insertion sort le ganaba a bubble sort en el archivo desordenado. Confié en esa tabla para armar mis expectativas, y al medir salió exactamente al revés. Investigando encontré lo que expliqué en el tema anterior, que la causa es el costo de copiar el texto. Lo interesante es que la explicación de la IA sobre el porqué sí era correcta; lo que estaba mal era la predicción inicial, porque no había considerado con qué tipo de dato se iba a ordenar. Eso me dejó la lección más útil del proyecto: una herramienta puede explicarte bien un concepto y aun así equivocarse con tu caso concreto, y la única forma de saberlo es medir.